

## UNIVERSITY OF CLAUDE · COURSE SYLLABUS

# CSCE 606 — Software Engineering

*Building software that survives contact with other people and with time*

## AT A GLANCE

**Credits:** 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 2 (with CSCE 611 and CSCE 645)

**Prerequisites:** Fluency in at least one language; experience with a codebase large enough to have become difficult

**Effort:** 8–10 hours per week · 13 modules + 2 project weeks

**Deliverable:** A codebase you have deliberately made maintainable: tested, reviewed, continuously integrated, instrumented, and documented well enough for someone else to take over

## Course description

Every other course in this program is about making a program *work*. This one is about making it keep working — after you have forgotten how it works, after three other people have modified it, and after the requirements have changed twice.

That sounds soft, and it is not. The economics are specific: most of the money spent on a piece of software is spent after it first ships, and most of that goes into understanding code before changing it. Every practice in this course is judged against that one measurement — does it reduce the cost of the next change?

The hazard in a subject like this is that it becomes a list of received opinions. So every claim here comes with the reasoning behind it and, where the evidence is weak, that is stated. Some widely repeated advice is wrong, or right only in particular circumstances, and the course says which.

This is also the course where the solo-study format is weakest, because several of its subjects — code review, collaboration, process — are inherently about working with other people. The projects are therefore built around open-source contribution, which supplies the other people.

## Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Distinguish essential from accidental complexity and attack the right one.
- 2 Elicit and specify requirements in a form that can be checked.
- 3 Design modules with low coupling and high cohesion, and explain why that reduces change cost.
- 4 Choose an architecture from the constraints that actually bind.

- 5 Design a test suite that gives confidence proportionate to its cost.
- 6 Refactor safely and reason about technical debt as a financial decision.
- 7 Review code in a way that improves it without damaging the author.
- 8 Build a pipeline that makes releasing boring.
- 9 Instrument a system so that its behaviour in production is observable.

## Module schedule

---

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

| Wk | Module                                       | Core question it answers                                                |
|----|----------------------------------------------|-------------------------------------------------------------------------|
| 1  | <b>Why Software Engineering Exists</b>       | What problem does software engineering solve that programming does not? |
| 2  | <b>Requirements and Specification</b>        | Why is it so hard to find out what people actually want?                |
| 3  | <b>Modularity: Coupling and Cohesion</b>     | What makes a codebase easy to change?                                   |
| 4  | <b>Architecture</b>                          | Which design decisions deserve to be made up front?                     |
| 5  | <b>Testing</b>                               | What should you test, and how much is enough?                           |
| 6  | <b>Test Design</b>                           | You cannot test everything. What do you test?                           |
| 7  | <b>Refactoring and Technical Debt</b>        | When is cleaning up code worth the time?                                |
| 8  | <b>Version Control and Collaboration</b>     | What is Git actually doing, and which workflow should you use?          |
| 9  | <b>Code Review</b>                           | What is review for, and how do you do it without damage?                |
| 10 | <b>Continuous Integration and Delivery</b>   | How do you deploy frequently without breaking things?                   |
| 11 | <b>Observability</b>                         | Something is wrong in production. How do you find out what?             |
| 12 | <b>Process</b>                               | Does the process matter, and which parts of it?                         |
| 13 | <b>Maintenance and the Long Life of Code</b> | How do you hand a system to someone who was not there?                  |

# Projects

---

## Project 1 — Make a codebase changeable (after Module 7)

Take a piece of your own code that has become hard to modify — ideally something from CSCE 641 or 645 — and make it changeable, measuring as you go.

### Requirements

- A written assessment of what makes it hard to change now, with specific examples rather than general complaints.
- A characterisation test suite covering current behaviour, written before any change.
- At least five refactorings, each a separate commit with a message explaining why.
- Coupling reduced demonstrably: a dependency diagram before and after.
- A written note on one refactoring you started and reverted, and why.

### Done means

- A change that was previously difficult, now made in under an hour, with the before-and-after effort recorded.
- Test suite runs in under a minute and fails if you break behaviour. Demonstrate by introducing a deliberate bug.
- A dependency diagram showing a reduction in cycles or in fan-out.

## Project 2 — Contribute to a real project (after Module 12)

Make a substantive contribution to an open-source project you did not write. The point is working inside someone else's constraints and surviving their review.

### Requirements

- A merged pull request of non-trivial size — not a typo fix. A bug fix with a regression test, or a small feature.
- Tests that match the project's existing conventions.
- A CI pipeline that passes, including any linting the project requires.
- The full review conversation, including every change requested.
- A written reflection on what the maintainers asked for that you had not anticipated.

### Done means

- The pull request is merged, or has a documented reason it was not.
- A record of review comments received, categorised by kind: correctness, style, design, project convention.
- An honest account of the most useful criticism you received.

## Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

| Component        | Weight | Standard                                                                                        |
|------------------|--------|-------------------------------------------------------------------------------------------------|
| Module exercises | 40%    | Code runs and produces the specified output. Not 'I understand how I would do it.'              |
| Projects         | 45%    | In the portfolio repository, with a README showing results and a written note on what was hard. |
| Self-checks      | 15%    | Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.  |

## Required materials

All free. Nothing in this course requires a purchase.

| Resource                                                                             | Role                                                                                                                                                  |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Software Engineering at Google (free online)</a>                         | The best single book on the subject, free from the authors. Its framing — engineering is programming integrated over time — is this course's framing. |
| <a href="#">MIT 6.031 Software Construction (free, complete)</a>                     | Excellent on specification, invariants, and testing. The readings are unusually precise.                                                              |
| <a href="#">Berkeley CS169 — Engineering Software as a Service (edX, free audit)</a> | A full project-based course, with the SaaS context that most software now lives in.                                                                   |
| <a href="#">Martin Fowler — refactoring.com and martinowler.com</a>                  | The reference for refactoring, continuous integration, and architecture patterns. Catalogue-style and genuinely useful.                               |
| <a href="#">Google Testing Blog and the Test Sizes taxonomy</a>                      | Practical testing writing from people running it at scale.                                                                                            |
| <a href="#">Accelerate / DORA research (summaries free)</a>                          | The one body of empirical evidence about software delivery practices. Read it before believing anything else in this field.                           |

## Tooling

- **Git**, used properly — interactive rebase, bisect, reflog, not just commit and push.
- **A CI service**: GitHub Actions is free for public repositories and sufficient for everything here.
- **A test framework** and a coverage tool for your language.
- **A linter and formatter**, configured once and then never argued about again.

- **A real open-source project** you will contribute to. Choose it in Module 01; it is the vehicle for both projects.

## Academic honesty, restated for a solo student

---

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.